InfiniLoRA: Disaggregated Multi-LoRA Serving for Large Language Models

Hongyu Chen
chenhongyu2048@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Letian Ruan
1291903308rlt@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Zilin Xu
xuzilin.200415@bytedance.com
Bytedance
China

Yuchen Li
yuchenli@smu.edu.sg
Singapore Management University
Singapore

Xinyu Chen
xinyuchen@hkust-gz.edu.cn
The Hong Kong University of Science and Technology (Guangzhou)
Guangzhou, China

Jingwen Leng
leng-jw@sjtu.edu.cn>
Shanghai Jiao Tong University
Shanghai, China

Bingsheng He
dcsheb@nus.edu.sg
National University of Singapore
Singapore

Minyi Guo
guo-my@cs.sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

Shixuan Sun
sunshixuan@sjtu.edu.cn
Shanghai Jiao Tong University
Shanghai, China

arXiv:2604.07173v1 [cs.DC] 8 Apr 2026

Abstract

LoRA enables efficient customization of LLMs and is widely used in multi-tenant and multi-task serving. However, emerging model architectures such as MoE significantly increase LoRA memory cost, making existing coupled LoRA serving designs poorly scalable and prone to tail-latency inflation. We present InfiniLoRA, a disaggregated LoRA serving system that decouples LoRA execution from base-model inference. InfiniLoRA introduces a shared LoRA Server with parallelism-aware execution, SLO-driven provisioning, and critical-path optimizations, including GPU-initiated communication and hardware-specialized LoRA kernels. Experiments show that InfiniLoRA can achieve an average 3.05× increase in serviceable request rate under strict latency SLOs, and improve the percentage of LoRA adapters satisfying the SLO requirement by 54.0%.

1 Introduction

Low-Rank Adaptation (LoRA) [6, 13, 29, 39] has become an important building block for deploying large language models (LLMs) in real-world systems. By enabling parameter-efficient fine-tuning, LoRA allows LLMs to incrementally incorporate task-specific, domain-specific, or user-specific knowledge without retraining or replicating the full model. This capability is particularly important for stateful applications, such as long-term memory [1, 4, 38], personalization [20, 22, 49], and preference modeling [2, 18, 45], where models are expected to retain, update, and apply behavioral or contextual information across interactions.

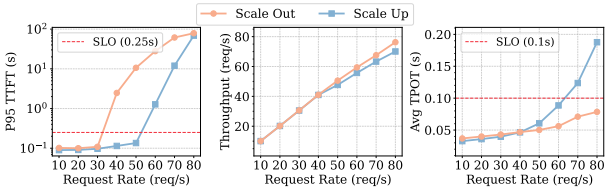
In cloud-based LLM serving platforms, these properties make LoRA well suited for multi-tenant scenarios. A single base model can be shared across many tenants or applications, while different LoRA adapters are dynamically activated to encode memories, skills, roles, or domain expertise on demand [3]. The serving frameworks further batch requests targeting different adapters to amortize computation and improve throughput [31, 40].

As LLM architectures continue to evolve, however, the assumptions underlying existing LoRA serving designs are increasingly strained. Emerging architectures, most notably Mixture-of-Experts (MoE) models [5, 8, 12, 16, 30, 32], significantly amplify the parameter footprint of LoRA adapters by introducing expert-specific adaptations as shown in Figure 1a. At the same time, the latest workloads often require longer context windows and richer interaction histories, substantially increasing the footprint of KV caches [9, 33, 44]. Together, these trends sharply reduce the effective capacity available for hosting LoRA adapters on GPUs, exposing fundamental scalability limits in current LoRA serving systems.

Limitations of Existing LoRA Serving Designs. Existing LoRA serving frameworks [3, 14, 21, 31, 40, 43, 46] adopt a *coupled design*, in which a working set of LoRA adapters is kept resident in GPU memory alongside the base model weights and KV cache to avoid adapter loading and queuing delays. This assumption largely holds for dense models, where each LoRA adapter is small. However, under MoE models, the design quickly breaks down. As shown in Figure 1a, due to the enlarged adapter footprint, only a small fraction of LoRA adapters can remain cached on GPUs. Requests targeting uncached adapters must therefore wait for in-flight executions to complete and free GPU memory before their adapters can be loaded, introducing significant queueing delays. As a result, tail Time-to-First-Token (TTFT), which is highly sensitive to queueing, is severely inflated.

Models	Lama3 8B	Qwen3 32B	Llama3 70B	GPT-OSS 20B	Mixtral 8x7B	Qwen3 30B A3B
#Layers	32	64	80	24	32	48
#Experts	N/A	N/A	N/A	32	8	128
Model (GB)	16	65	140	40	94	60
KV Cache (GB)	0.16	0.25	0.31	0.05	0.16	0.09
Single LoRA (GB)	0.21	0.70	1.05	1.58	1.69	6.18
#LoRAs per 100GB	474	142	95	63	59	16

(a) Memory consumption of model weights, KV cache (1024 tokens), and LoRA for representative dense and MoE models, with LoRA rank=64.



(b) Scale-out (Eight 2-GPU Instances) vs. Scale-up (Four 4-GPU Instances) performance comparison under varying Load on Nvidia Hopper GPUs using Mixtral 8x7B models and 512 LoRAs.

Figure 1. (**Top**) LoRA cache capacity across model architectures. (**Bottom**) Scale-out vs. scale-up performance.

A natural response is to increase LoRA cache capacity by either *scaling out*, i.e., deploying more LLM instances, or *scaling up*, i.e., allocating more GPUs to a single instance. However, both approaches have fundamental limitations. Scaling out increases total cache capacity across instances but requires duplicating base model weights and KV caches for each instance, incurring substantial GPU memory overhead. Given the size of modern LLMs, this duplication leaves little additional memory for LoRA, resulting in only marginal cache gains. As shown in Figure 1b, P95 TTFT quickly degrades and violates the TTFT SLO even when using eight 2-GPU instances, due to limited effective cache capacity.

Scaling up aggregates more GPUs within a single instance, expanding the LoRA cache without duplicating base weights. However, it enlarges the communication scope: as the instance spans more GPUs, especially across nodes, communication overheads grow rapidly and inflate inference latency. Moreover, efficiently utilizing the increased resources requires larger batch sizes, which are often incompatible with latency-sensitive workloads. Consequently, as shown in Figure 1b, scaling up achieves lower throughput than scaling out and suffers significantly higher average Time-Per-Output-Token (TPOT). In summary, the core limitation of existing approaches stems from a *coupled* design that tightly binds LoRA adapters to the base model within the individual LLM instances. As a result, accommodating dynamic LoRA workloads requires modifying base-model execution, which limits flexibility and leads to inefficient resource utilization.

Our Approach. To overcome these limitations, we propose a *disaggregated LoRA serving architecture* that allows LoRA adapters to be shared across multiple LLM instances. LoRA adapters are managed and executed by a dedicated LoRA Server, while LLM instances remain LoRA-free and focus on base-model inference. This decoupling enables LoRA resources to scale independently of base-model execution, but also introduces new challenges in placing, executing, and coordinating LoRA computation across the system.

Challenge 1: Parallelism Design after Disaggregation. In coupled designs, LoRA execution implicitly follows the base model's parallelism strategy. Disaggregation breaks this assumption: once LoRA adapters are decoupled, their placement directly determines how LoRA computation is parallelized. The system must therefore explicitly design how LoRA computation is partitioned, synchronized, and scaled across GPUs.

Challenge 2: Preserving the Inference Critical Path. LoRA computation lies on the decode-time inference path and directly affects TPOT. Disaggregating LoRA introduces additional communication, synchronization, and adapter loading into inference. Without careful optimization, these overheads can extend the critical path. The key challenge is to offload LoRA execution without increasing inference latency.

Challenge 3: SLO-Driven Resource Provisioning. LoRA access patterns are highly dynamic. Insufficient LoRA Server capacity leads to queueing and tail-latency violations, while over-provisioning wastes resources. Accurately provisioning LoRA Server resources to meet both TTFT and TPOT SLOs thus becomes a fundamental system challenge.

To address these challenges, we present **InfiniLoRA**, a parallelism-aware, SLO-driven, and critical-path–optimized LoRA serving system. First, InfiniLoRA pipelines each adapter request across receive–compute–send stages to absorb concurrency from multiple LLM instances, and adopts a hybrid execution strategy that combines expert parallelism with pipeline parallelism to balance synchronization overhead, GPU utilization, and communication granularity.

Second, InfiniLoRA employs a SLO-driven resource provisioning to handle the dynamic LoRA workloads. Based on the transformation of the service's TTFT SLO attainment rate into the probability of requests being immediately admitted, along with historical LoRA invocation information, it employs binary search and dynamic programming algorithms to determine the minimal LoRA cache capacity, ultimately deducing the minimum GPU requirement for LoRA Server.

Third, to minimize LoRA processing overhead, InfiniLoRA leverages host-bypass, GPU-initiated communication with a push-based protocol to reduce network latency on the critical path. It further integrates hardware-specialized LoRA kernels that exploit modern GPU features, improving LoRA computation effciency. To mitigate cold-start overhead, InfiniLoRA pipelines adapter loading with execution and performs scheduler-driven prefetching before the first LoRA invocation.

Using request streams derived from both production traces and synthetic workloads, our evaluation shows that InfiniLoRA significantly outperforms state-of-the-art multi-LoRA serving systems. InfiniLoRA can sustain an average 3.05× higher request rates under strict TTFT and TPOT SLOs. From a multi-tenant perspective, it boosts the ratio of LoRA adapters meeting stringent service-quality targets (i.e., over 90% of requests satisfying SLOs) by 53.1%.

2 Background

2.1 LLM Inference

LLM inference consists of two phases: *prefill* and *decode*. In the prefill phase, the entire input prompt is processed in parallel to initialize the KV cache. This phase is dominated by dense matrix multiplications and is therefore compute-bound. In contrast, the decode phase generates tokens autoregressively, one token per step, repeatedly reading and writing the KV cache. As a result, decode execution is memory-bandwidth-bound and exposes limited parallelism per request. When prefill and decode are co-located on the same GPUs, prefill requests can monopolize GPU execution and delay decode steps, directly inflating both TTFT and TPOT. To avoid this interference, recent serving systems [10, 11, 25, 26, 50] adopt a prefill–decode disaggregated architecture (Figure 2), assigning the two phases to separate resources. In practice, a prefill GPU can reach high utilization with a very small batch size, often a single request, due to its high arithmetic intensity, whereas decode GPUs require batching tens to hundreds of concurrent requests to saturate memory bandwidth and achieve high throughput.

During decoding, large models are executed using hybrid parallelism [27, 48]. For MoE models, *expert parallelism* partitions experts across multiple GPUs, while attention layers are executed with *data parallelism*, where the input batch is sharded across GPUs and routed to the corresponding experts. If needed, *tensor parallelism* further splits attention computation across devices. These parallelization strategies are carefully optimized to balance inference latency, throughput, and cost efficiency.

2.2 LoRA Computation and Multi-LoRA Serving

As shown in Figure 3, for a weight matrix $W \in \mathbb{R}^{h\times d}$, LoRA applies a low-rank update such that $W' = W + AB$, where $A \in$

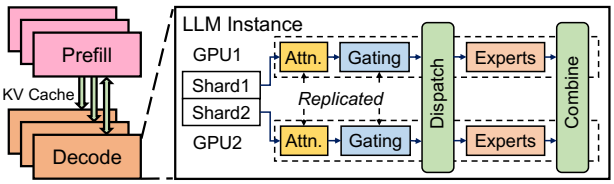
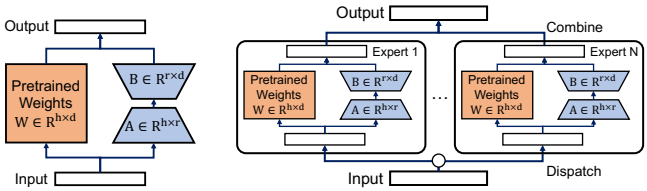


Figure 2. Prefill–decode disaggregated architecture. LLM instances are deployed with 2 GPUs using expert parallelism.



(a) Dense model. **(b)** MoE model.

Figure 3. LoRA computation on Dense and MoE models.

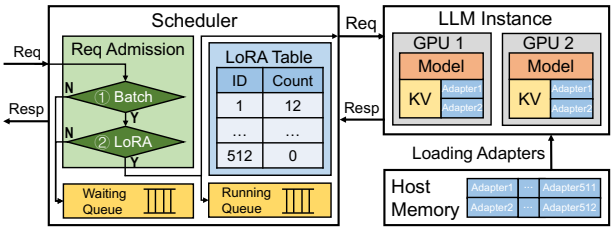


Figure 4. Coupled-design multi-LoRA serving architecture.

$\mathbb{R}^{h\times r}$ and $B \in \mathbb{R}^{r\times d}$ are trainable matrices, called *adapters*. Given an input x, the output becomes $y' = xW' = xW + xAB$. The rank r is typically small (e.g., 32–128), which significantly reduces both training cost and inference overhead compared to fully fine-tuning [6, 13, 28].

In multi-task and multi-tenant serving environments, many LoRA adapters will be served concurrently, and different requests within the same batch may require different adapters. To support this, recent systems [3, 14, 19, 21, 31, 40, 46] exploit the fact that multiple LoRA adapters are derived from a shared base LLM and enable multi-LoRA serving by consolidating such requests into a single inference batch. Specifically, the base model computation (xW) is performed in a batched manner, while each request independently computes its corresponding LoRA update (xAB), which is then added to the base output. Existing systems adopt a *coupled architecture* in which LoRA adapters are stored in GPU memory alongside the base model weights and KV cache within each LLM engine. As shown in Figure 4, these systems employ a LoRA-aware execution flow to schedule incoming requests.

Upon receiving a request, the scheduler first verifies if the engine has reached its maximum batch size, constrained by KV cache capacity and TPOT SLOs. If within limits, it consults the LoRA table. The request is admitted if the required adapter is already resident or can be loaded into available cache space; otherwise, it is queued. Scheduling occurs at the token level, allowing requests to be admitted or retired at each decoding step, while dynamically updating the LoRA table.

Under the coupled architecture, prior work primarily improves LoRA serving performance through optimizations in the following aspects. These include optimizing cache replacement policies to increase LoRA cache hit rates [14, 41,

53], redesigning data layouts to reduce adapter loading overhead [21, 34], rebalancing GPU memory allocation between KV cache and LoRA cache [46], and developing specialized GPU kernels to accelerate LoRA computation when requests within a batch require different adapters that scattered across memory [17, 42, 51].

LoRA serving during the prefill stage is relatively straightforward. The batch size is typically small (e.g., 1–4 requests) [7, 50], limiting the GPU memory overhead of LoRA caching. In addition, the prefill cost is easy to estimate because input sequence lengths are known in advance, allowing required LoRA adapters to be prefetched by overlapping adapter loading with the computation of previous batches. In contrast, LoRA serving during the decode stage is substantially more challenging. Decode batches are larger, and the number of decoding steps per request is difficult to predict. Consequently, both prior work and our design focus mainly on optimizing LoRA serving in the decode phase.

2.3 MoE-Induced Issues in Multi-LoRA Serving

As LLMs evolve, the MoE architecture has become increasingly prevalent and is adopted by recent models such as Mixtral [16], Qwen3 [35], and DeepSeek [5]. In MoE models, LoRA adapters maintain expert-specific parameters, causing adapter size to scale with the number of experts (Figure 3b). As the expert count grows, the memory footprint of each adapter increases substantially. Consequently, under a fixed LoRA memory budget, the number of adapters that can reside on GPUs drops sharply (Figure 1a), significantly undermining the effectiveness of multi-LoRA serving.

To quantify this effect, we evaluate S-LoRA [31], using its state-of-the-art implementation integrated into vLLM [19], as opposed to the original vanilla codebase, on the Mixtral 8x7B model. We deploy four LLM instances, each running on two NVIDIA Hopper GPUs (96 GB), and vary the LoRA cache ratio from 10% to 50%, where the cache ratio denotes the fraction of adapters that can be simultaneously resident in GPU memory across all instances. The system serves 256 LoRA adapters, each consuming 1.69 GB of GPU memory, with access frequencies following a Zipf distribution ($s = 1.2$) as in prior work [3, 31]. The P95 TTFT SLO is set to 0.25 s. This setup isolates the impact of limited LoRA cache capacity in coupled architectures. Our results reveal two key issues that arise when cache capacity is constrained.

Issue 1: a low cache capacity leads to excessive TTFT[1], directly degrading service quality. When the required LoRA adapter is not resident in GPU memory, incoming requests must wait until in-flight executions complete and GPU memory becomes available to load its adapter. This introduces additional queuing and loading delays before decoding can begin. Since TTFT is highly sensitive to such delays, even moderate cache misses can significantly inflate tail latency, resulting in poor SLO compliance. As shown in Figure 5, the P95 TTFT reaches hundreds of seconds when the LoRA cache ratio (defined as the cache capacity divided by the total number of LoRAs) is low. As cache ratio increases, the P95 TTFT drops sharply. Further analysis of adapter-level compliance reveals a consistent trend: a larger cache ratio allows significantly more LoRA adapters to exceed high SLO satisfaction thresholds (e.g., >80% or >90% of their requests).

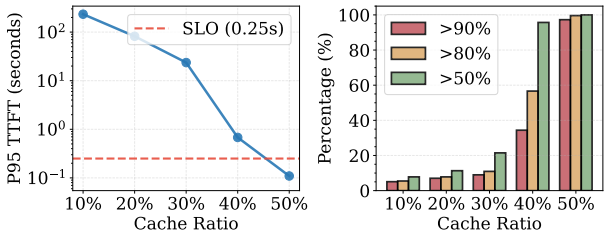


Figure 5. Impact of LoRA cache ratio on TTFT performance and SLO attainment. (**Left**) P95 TTFT under varying cache ratios, with SLO of 0.25 seconds. (**Right**) Percentage of LoRA adapters for which the fraction of requests meeting the TTFT SLO exceeds specific thresholds (50%, 80%, and 90%).

Issue 2: an insufficient cache capacity reduces the effective batch size, lowering hardware utilization. Requests targeting uncached LoRA adapters cannot be admitted into the execution engine and must remain in the waiting queue. As a result, the engine operates with fewer requests than its configured batch size limit, even when sufficient compute resources are available. This underutilization is particularly detrimental during the decode phase, which requires large batch sizes to saturate memory bandwidth. Figure 6 illustrates the batch size observed over time. Request arrivals follow a Poisson process, causing the batch size to fluctuate dynamically. When cache ratio is low, the batch size remains consistently small because only a limited number of adapters can reside on GPUs at any time. Moreover, the batch size exhibits high variance because request admission is strictly constrained by the currently resident adapters.

As discussed in Section 1, neither scaling out nor scaling up can fundamentally resolve the limitations of LoRA serving under the *coupled* architecture. Scaling out suffers from excessive duplication of base model parameters and isolated LoRA caches, leading to poor memory efficiency. Scaling up avoids parameter duplication but expands the communication scope and requires larger batch sizes, which increases inference overhead and inflates TPOT. Despite their differences, both approaches are constrained by the same root cause: the coupled architecture tightly binds LoRA

[1]In multi-LoRA serving with PD disaggregation, we focus on the latency of generating the first token in the decode phase, as discussed in Section 2.1. Accordingly, we redefine TTFT for a request as the sum of its queueing delay and the time required by the decoding engine to produce the first output token, explicitly excluding the prefill phase. This definition isolates the performance impact of decode-time LoRA serving and directly reflects user-perceived service quality.

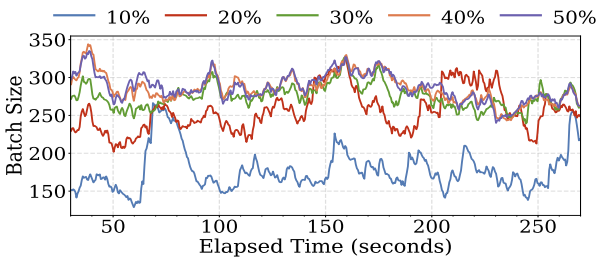


Figure 6. Impact of LoRA cache ratio on batch size. Measurements are collected during the steady-state interval (30–270s) of the 300s experiment.

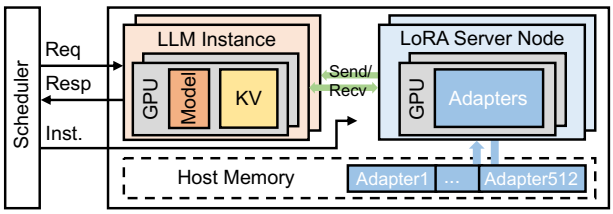
adapters to the base model execution. This tight coupling forces dynamic LoRA adapters to scale in lockstep with the heavyweight base model, limiting scalability and efficiency.

3 An Overview of InfiniLoRA

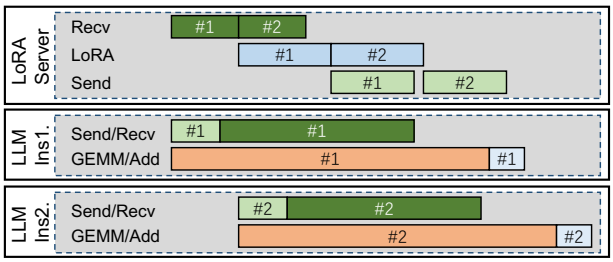
To overcome the fundamental limitations of the coupled architecture, we decouple LoRA adapters from LLM instances, enabling adapters to be shared across multiple instances and allowing LoRA cache capacity to scale independently of base-model execution. To this end, we propose **InfiniLoRA**, a LoRA serving system built on a disaggregated architecture.

Figure 7 illustrates the architecture and execution workflow of InfiniLoRA. Unlike coupled designs, InfiniLoRA manages LoRA adapters in a dedicated LoRA Server, which may span multiple nodes, while LLM instances remain LoRA-free and execute the base model using their existing optimization strategies. During request processing, an LLM instance performs base-model computation and forwards the corresponding activations to the LoRA Server. The LoRA Server applies the requested LoRA computation and returns the updated activations, which are then integrated back into the LLM inference pipeline. This procedure is performed twice within each MoE layer, corresponding to the fine-tuned up-gate and down-projection matrices. It is worth noting that, while the LoRA Server executes LLM instance requests one by one instead of fusing them into a single batch, it still achieves concurrency across different instances by pipelining communication and computation. This disaggregated design allows LoRA caching and computation to scale independently of base-model execution and avoids interference with the highly optimized LLM inference pipeline.

Realizing this disaggregated design requires addressing three key system design aspects. First, the system must define an explicit LoRA parallel execution strategy after disaggregation, which determines how LoRA computation is placed, synchronized, and scaled across GPUs. Second, remote LoRA execution must be carefully integrated into the decode-time inference path to minimize the impact on the critical path and avoid degrading TTFT and TPOT. Third, LoRA Server resources must be provisioned in an SLO-aware manner to



(a) Disaggregated architecture.



(b) Execution workflow: LLM instances send activations to the LoRA Server and receive the computed results. This communication is overlapped with local GEMM computations, followed by a final addition of the two outputs.

Figure 7. An overview of InfiniLoRA.

balance latency guarantees and resource efficiency under dynamic and skewed workloads. InfiniLoRA addresses these aspects through parallelism-aware LoRA execution and SLO-driven resource provisioning in Section 4, and critical-path optimization in Section 5.

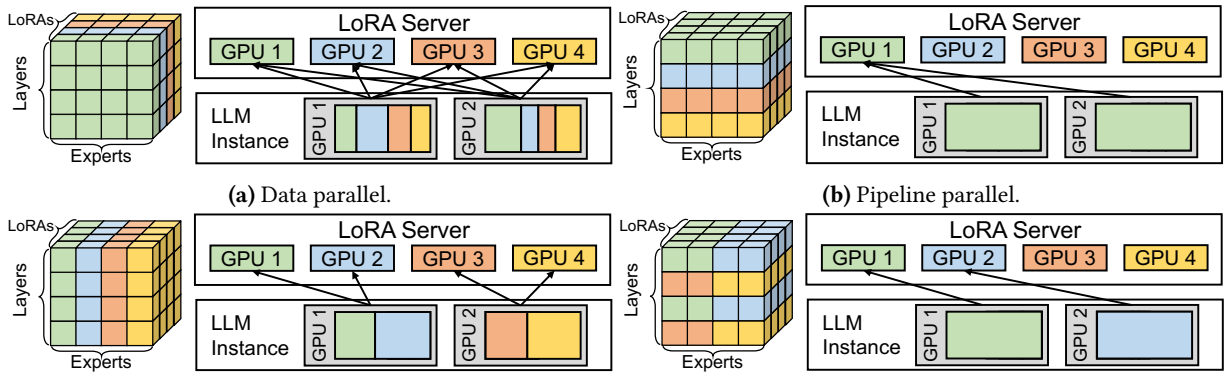
4 LoRA Server Design: Parallelism-Aware Execution and SLO-Driven Provisioning

This section presents the core LoRA Server design of InfiniLoRA, addressing two key aspects: parallelism-aware LoRA execution (Section 4.1) and SLO-driven resource provisioning (Section 4.2).

4.1 Parallelism-Aware LoRA Execution

Given an MoE model with l layers and e experts per layer, suppose there are n LoRA adapters and the LoRA Server contains m GPUs. We abstract the LoRA adapters as a three-dimensional tensor of size $n \times l \times e$, where each unit corresponds to the LoRA matrix associated with a specific expert at a specific layer of a given adapter (Figure 8). For brevity, we denote the GPUs of the LLM instance and the LoRA Server as *client* and *server* GPUs, respectively.

Consider an LLM instance with p GPUs and batch size b, using expert parallelism with degree p. Each request activates the top-k experts at each layer, so the batch activates $b \times k$ LoRA computation units per layer. For each activated expert, the corresponding activation is sent to the LoRA Server, processed, and returned. Therefore, the total activation volume transferred from the LLM instance to the LoRA Server for a layer is $b \times k$. We analyze four parallelization strategies



(a) Data parallel.

(b) Pipeline parallel.

(c) Expert parallel.

(d) Hybrid parallel.

Figure 8. LoRA adapter placement strategies across server GPUs. The three-dimensional block represents the adapter space, with axes corresponding to LoRA adapters, layers, and experts. Each color indicates the server GPU (GPU 1–4) to which a partition of adapters is assigned. Arrows depict the activation data flow between client GPUs and server GPUs.

using four metrics summarized in Table 1: *Peer Comm Volume* (average activation transferred per client–server GPU pair), *Comm Peer Count* (number of client GPUs communicating with a server GPU), *Compute Volume* (LoRA computation per server GPU), and *Sync Scope* (number of server GPUs that must synchronize per step).

LoRA Data Parallel. A straightforward approach is to evenly distribute LoRA adapters across the server GPUs, as illustrated in Figure 8a. Because requests within a batch may access different adapters residing on any server GPU, activations from client GPUs must be routed accordingly. When multiple LLM instances share a LoRA Server, the server GPUs perform a collective coordination step to determine which activations should be processed by which GPUs. As a result, the communication peer count for a server GPU is at most p, and the sync scope spans all m server GPUs.

On average, one client GPU can produce $\frac{b \times k}{p}$ expert activations, each of which must be sent to the server GPU hosting the corresponding adapter. With adapters evenly distributed across m server GPUs, these activations are further spread, yielding an average pairwise peer comm volume of $\frac{b \times k}{p \times m}$ and an average compute volume of $\frac{b \times k}{m}$ per server GPU. In practice, expert load imbalance can skew the number of activations per client GPU, while skewed adapter access patterns can concentrate activations on a subset of server GPUs, increasing communication and computation imbalance despite uniform adapter placement.

LoRA Pipeline Parallel. As shown in Figure 8b, an alternative approach is to apply layer parallelism by organizing LoRA adapters by layer and assign each layer's complete n adapters to a single server GPU, evenly distributing the l layers across the m server GPUs.[2] Under this design, when processing a given layer, all client GPUs within a LLM instance send their activations to the same server GPU that hosts the corresponding layer, so the comm peer count is p and the sync scope is 1 and yielding a pairwise peer communication volume of $\frac{b \times k}{p}$ and a compute volume of $b \times k$ on the server GPU.

Table 1. Comparison of LoRA execution parallelization strategies. b, k, p, m: per-instance batch size, expert routing top-k, GPU counts for an LLM instance and the LoRA Server. x, y: hybrid parallelism configuration.

Parallel Strategy	Comm Peer Volume	Comm Peer Count	Compute Volume	Sync Scope
DP	$\frac{b \times k}{p \times m}$	m	$\frac{b \times k}{m}$	m
PP	$\frac{b \times k}{p}$	1	$b \times k$	1
EP	$\frac{b \times k}{\max(p,m)}$	$\max(\frac{m}{p}, 1)$	$\frac{b \times k}{m}$	m
EP_x-PP_y	$\frac{b \times k}{\max(p,x)}$	$\max(\frac{x}{p}, 1)$	$\frac{b \times k}{x}$	x

Because all experts and adapters for a layer reside on the single server GPU, this approach avoids load imbalance across server GPUs for that layer. Compared to data-parallel LoRA execution, the synchronization overhead is minimal, and different LLM instances can process LoRA computation for different layers concurrently on different server GPUs. However, concentrating all LoRA computation for a layer on a single server GPU places heavy communication and computation load on the decode-time critical path, potentially degrading inference latency.

LoRA Expert Parallel. As shown in Figure 8c, we organize LoRA adapters by expert and evenly distribute the e experts' adapters across the m server GPUs, so that each server GPU hosts adapters for $\frac{e}{m}$ experts. For a given layer, any expert

[2] A single GPU can typically hold all adapters for a given layer, since each LoRA adapter occupies only several MBs. If the total adapter size for a layer exceeds a single GPU's capacity, multiple GPUs can be grouped to host that layer. For simplicity, we assume one GPU suffices per layer.

may be activated by the batch, so the sync scope spans all m server GPUs. Using aligned expert partitioning, each client GPU owns $\frac{e}{p}$ experts, and each server GPU receives activations only from the client GPUs with same experts, limiting the communication peer count to $\max\left(\frac{p}{m}, 1\right)$. On average, the $b \times k$ expert activations per layer are evenly spread across the m server GPUs, yielding a compute volume of $\frac{b \times k}{m}$ per server GPU and a pairwise peer communication volume of $\frac{b \times k}{\max(p,m)}$. Compared with pipeline execution, expert parallelism avoids concentrating a layer's LoRA computation on a single GPU, but incurs a larger sync scope, fragmented communication and may suffer from load imbalance due to skewed expert activation patterns.

LoRA Hybrid Parallel. Naturally, we should balance three competing objectives: keeping the sync scope small to reduce synchronization overhead, avoiding overly fragmented communication and computation across server GPUs, and minimizing the impact on the inference critical path.

Motivated by this trade-off, we propose a hybrid parallelism scheme that combines pipeline and expert parallelism. We denote a hybrid configuration as EP_x-PP_y, where x is the degree of expert parallelism, y is the number of pipeline stages, and $x \times y = m$. As illustrated in Figure 8d, we set $x = 2$ and $y = 2$, partitioning the server GPUs into two pipeline stages, each with expert parallelism degree two. Rather than grouping contiguous layers within a stage, we interleave layers across GPU groups (e.g., assigning Layers 1 and 3 to GPUs 1–2, and Layers 2 and 4 to GPUs 3–4) to reduce LoRA loading overhead, discussed in Section 5.3.

Under hybrid parallelism, LoRA computation for a layer involves only the x GPUs within its expert group; therefore, the sync scope is x. Each server GPU receives activations from at most $\max\left(\frac{p}{x}, 1\right)$ client GPUs, yielding a comm peer count of $\max\left(\frac{p}{x}, 1\right)$. The compute volume per server GPU is $\frac{b \times k}{x}$, and the average pairwise comm peer volume is $\frac{b \times k}{\max(p,x)}$. By tuning x and y, hybrid parallelism enables flexible trade-offs among these metrics. Increasing x reduces per-GPU compute volume by involving more server GPUs, but also increases synchronization overhead, fragments communication, and amplifies the impact of expert load imbalance. Conversely, smaller x reduces synchronization cost but concentrates computation. We empirically tune x and y offline; in practice, setting x equal to the number of intra-node GPUs is generally a good default, as prioritizing a larger x is more beneficial for efficiency (Section 6.3.2).

4.2 SLO-Driven LoRA Server Resource Provisioning

We formulate provisioning based on two inputs: (1) LoRA workload characteristics, assuming historical access patterns are recurrent [53]; and (2) LLM instance load, parameterized by batch size. The system must jointly satisfy tail (P95) TTFT and average TPOT SLOs. Table 2 summarizes the notation.

Table 2. Key notation for Section 4.2.

Symbol	Description
N	Total number of LoRA adapters
L	Number of LLM instances
B	Batch size per LLM instance
LB	Global batch size, i.e., $L \cdot B$
M	LoRA cache capacity
p_i	Request-level invocation probability of adapter i
λ_i	Expected access count of adapter i in a global batch, i.e., $LB \cdot p_i$
τ^*	Admission threshold for the Poissonized model
q_i	Residency (cache) probability of adapter i
$P_{\text{free}}(i)$	Probability that a free cache slot exists for adapter i
$\text{IAR}(M)$	Immediate Admissibility Rate under cache capacity M
α	Target Immediate Admissibility Rate (e.g., 0.95)
Mem_{LoRA}	GPU memory footprint per adapter

4.2.1 Satisfying the Tail TTFT SLO.

From TTFT to Immediate Admissibility. In low-latency inference systems, request queuing is the dominant cause of TTFT SLO violations. Consequently, meeting a tail TTFT SLO reduces to ensuring that a vast majority of requests bypass queuing. We formalize this requirement through the *Immediate Admissibility Rate* (IAR). Specifically, a request is deemed *immediately admissible*—meaning it incurs zero queuing delay—if its target LoRA adapter is either already resident in GPU memory or can be instantly loaded into an available slot. Under this formulation, satisfying P95 TTFT SLO is equivalent to maintaining an IAR of at least 95%.

Problem Formulation. Given the invocation probability distribution $\{p_i\}_{i=1}^{N}$, the global batch size LB, and a target Immediate Admissibility Rate $\alpha \in (0, 1]$, our goal is to find the minimum cache capacity:

$$M^* = \min \left\{ M \in [N] \;\middle|\; \text{IAR}(M) \geq \alpha \right\} \tag{1}$$

We next develop a probabilistic model and corresponding search algorithm to find this M^*.

Probabilistic Modeling. We model the system in a steady state where the LoRA Server maintains a working set of M resident adapters. For each adapter i, its expected access count within a global batch is $\lambda_i = LB \cdot p_i$. We adopt a Poissonized model: the actual access count of adapter i in a gloabl batch is treated as $\text{Poisson}(\lambda_i)$. And an adapter is considered *resident* if its access count exceeds an admission threshold τ^*.

The *residency probability* of adapter i is therefore:

$$q_i \;=\; \Pr\big[\text{Poisson}(\lambda_i) > \tau^*\big] \;=\; 1 - \sum_{k=0}^{\tau^*} \frac{\lambda_i^k \, \exp(-\lambda_i)}{k!} \tag{2}$$

Then the threshold τ^* can be uniquely determined by the capacity constraint:

$$\sum_{i=1}^{N} q_i \;=\; M \tag{3}$$

which states that the expected number of resident adapters—each independently present with probability q_i—exactly fills the cache capacity.

For any incoming request, the probability that it targets adapter i and does *not* require queuing decomposes into two mutually exclusive cases: *(i)* a direct cache hit (probability q_i), or *(ii)* a cache miss but a free slot exists among the M positions. Let $P_{\text{free}}(i)$ denote the probability that the remaining $N-1$ adapters collectively occupy at most $M-1$ slots. Since each adapter $j \neq i$ independently resides in the cache with probability q_j, the total number of occupied slots is a sum of $N-1$ independent Bernoulli random variables. $P_{\text{free}}(i)$ is then the probability that this sum does not exceed $M-1$. Aggregating over all adapters, the overall IAR is:

$$\text{IAR}(M) \;=\; \sum_{i=1}^{N} p_i \left[\, q_i \;+\; (1-q_i) \;\cdot\; P_{\text{free}}(i) \,\right] \tag{4}$$

Solution Procedure. Algorithm 1 solves Problem (1) by incrementally testing $M = 1, 2, \ldots$ and returning the first value that satisfies the IAR target. For each candidate M, the algorithm proceeds in three stages. First, it solves Eq. (3) for τ^* via binary search (line 3) and obtains all residency probabilities $\{q_i\}$ (line 4). Second, for each adapter i, it uses dynamic programming to compute $P_{\text{free}}(i)$ and accumulates the per-adapter contribution to the IAR via Eq. (4) (line 15). Finally, it checks whether $\text{IAR}(M) \geq \alpha$ (line 17) and, if so, returns $M^* = M$.

Deriving the Minimum Cache Size. Given the IAR formulation established above, we search for the smallest cache capacity M such that $\text{IAR}(M) \geq \alpha$, where α corresponds directly to the tail TTFT SLO target (e.g., $\alpha = 0.95$ for a P95 requirement). The minimum GPU memory budget for the LoRA Server is then $M^* \times \text{Mem}_{\text{LoRA}}$.

Algorithm 1 Minimum Cache Size for Tail TTFT SLO

Require: LoRA invocation probabilities $\{p_i\}_{i=1}^{N}$, global batch size LB, target IAR α
Ensure: Minimum cache size M^*

1: Compute $\lambda_i \leftarrow LB \cdot p_i$ for all $i \in [N]$
2: **for** $M = 1$ **to** N **do**
3: $\quad \tau^* \leftarrow \textsc{BinarySearch}(\tau \text{ s.t. } \sum_{i=1}^{N} Q(\lambda_i, \tau) = M)$ ▷ $Q(\lambda, \tau) = \Pr[\text{Poisson}(\lambda) > \tau]$
4: $\quad q_i \leftarrow Q(\lambda_i, \tau^*)$ for all i
5: $\quad \text{IAR} \leftarrow 0$
6: $\quad$ **for** $i = 1$ **to** N **do**
7: $\quad\quad \text{dp}[0] \leftarrow 1; \;\; \text{dp}[1..N] \leftarrow 0$ ▷ DP over Poisson-Binomial
8: $\quad\quad$ **for** each $j \in [N] \setminus \{i\}$ **do**
9: $\quad\quad\quad$ **for** $k = N-1$ **down to** 1 **do**
10: $\quad\quad\quad\quad \text{dp}[k] \leftarrow \text{dp}[k] \cdot (1-q_j) + \text{dp}[k-1] \cdot q_j$
11: $\quad\quad\quad$ **end for**
12: $\quad\quad\quad \text{dp}[0] \leftarrow \text{dp}[0] \cdot (1-q_j)$
13: $\quad\quad$ **end for**
14: $\quad\quad P_{\text{free}}(i) \leftarrow \sum_{k=0}^{M-1} \text{dp}[k]$
15: $\quad\quad \text{IAR} \leftarrow \text{IAR} + p_i \cdot \big[q_i + (1-q_i) \cdot P_{\text{free}}(i)\big]$
16: $\quad$ **end for**
17: $\quad$ **if** $\text{IAR} \geq \alpha$ **then**
18: $\quad\quad$ **return** $M^* \leftarrow M$
19: $\quad$ **end if**
20: **end for**

4.2.2 Satisfying the Average TPOT SLO.

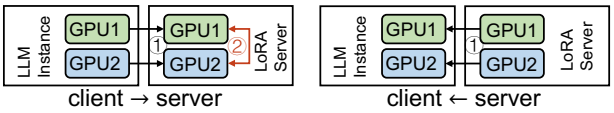
We next address the average TPOT SLO by deriving the computation and communication resources required by the LoRA Server. We formulate the hardware resource constraints from two perspectives. From the perspective of the *LLM instance*, the global average TPOT SLO imposes a latency constraint:

$$T_{\text{recv}} + T_{\text{comp}} + T_{\text{send}} \;\leq\; \text{SLO}_{\text{FFN}} \tag{5}$$

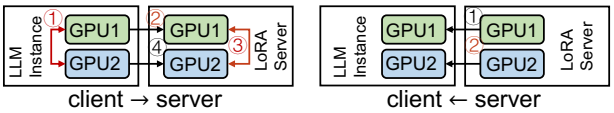
where T_{recv}, T_{comp}, and T_{send} denote the latencies incurred by the LoRA Server to receive activations, execute LoRA computation, and return the results for a single LLM instance, respectively. All three terms can be profiled and modeled as functions of the per-instance batch size B. Additionally, SLO_{FFN} represents the latency budget allocated to the experts' Grouped GEMM modules, which is derived from the global average TPOT SLO. From the *LoRA Server* perspective, the LoRA Server must serve all L LLM instances within the time window of a single layer, yielding the throughput constraint:

$$\max(T_{\text{recv}},\; T_{\text{comp}},\; T_{\text{send}}) \;\cdot\; L \;\leq\; \text{SLO}_{\text{Layer}} \tag{6}$$

where $\text{SLO}_{\text{Layer}}$ denotes the total latency budget for one base model layer, encompassing both the attention and expert modules. By jointly solving Eq. (5) and Eq. (6), we derive the minimum number of LoRA Server GPUs required to satisfy the average TPOT SLO. Combined with the cache capacity M^* obtained from Algorithm 1 for the tail TTFT SLO, the overall minimum GPU count of the LoRA Server is fully determined by the two SLO targets.



(a) Push-based communication.



(b) Pull-based communication.

Figure 9. Design for client–server communication. Control signals are labeled red and data transfers are labeled black.

5 Critical-Path Optimization for Disaggregated LoRA Execution

This section presents the communication and LoRA computation kernel designs required to make disaggregation practical, along with optimizations for LoRA loading.

5.1 Host-Bypass Client-Server GPU Communication

Although LoRA computation is lightweight, disaggregated execution is often bottlenecked by communication due to the large bandwidth gap between GPU HBM (4.0 TB/s on Hopper 96 GB) and inter-GPU links (50 GB/s over InfiniBand versus 900 GB/s over NVLink). Effectively overlapping LoRA execution with base-model computation therefore requires novel communication design.

Unlike collective communication in multi-GPU LLM inference, communication between LLM instances (clients) and the LoRA Server follows a dynamic client–server pattern, where request to process LoRA arrive asynchronously from different instances. This precludes synchronous NCCL P2P primitives, which require sender–receiver rendezvous; the LoRA Server cannot determine the next sender in advance to issue a matching `ncclRecv`. We therefore implement client–server communication using `IBGDA` (InfiniBand GPUDirect Async) [23], which supports one-sided RDMA operations directly from GPU kernels. We support both *push-based* and *pull-based* modes; below we describe the client-to-server path as a representative example.

Push-based Mode. Client GPUs directly write activations into preallocated buffers on target server GPUs via one-sided RDMA writes (①). A leader server GPU runs a persistent kernel that polls these buffers and, upon detecting a completed write, broadcasts the client ID to other server GPUs to coordinate processing (②). This avoids sender–receiver synchronization and enables low-latency batch admission.

Pull-based Mode. In contrast, pull-based communication requires clients to first coordinate locally (①) and notify the server (②), after which server GPUs explicitly synchronize locally (③) and issue remote reads (④), introducing extra synchronization overhead and network round trips. Our measurements show that for typical payloads (e.g., 4 MB), pull-based communication incurs 2.63× higher latency than push-based.

A similar asymmetry applies to the server-to-client path: pull-based designs require clients to repeatedly poll remote completion states, incurring additional round-trip latency. Consequently, InfiniLoRA adopts push-based communication in both directions to minimize decode-time latency.

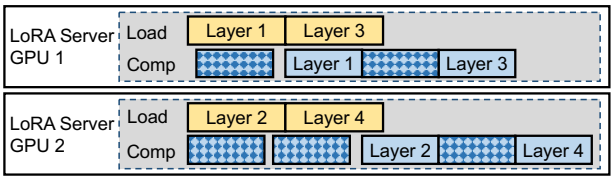


Figure 10. Layer-wise LoRA loading. Shaded blue blocks represent LoRA execution from any other LLM instance.

5.2 Hardware-Specialized LoRA Kernels

LoRA computation differs fundamentally from GEMM: it must gather scattered activations from non-contiguous memory and perform fine-grained GEMM/GEMV operations, yielding performance characteristics that diverge significantly from dense matrix multiplication. Since communication already occupies part of the critical path, LoRA computation can easily become a decode-time bottleneck without careful kernel optimization. Building on SGMV [3] and BGMV [31], we design hardware-specialized LoRA kernels for modern GPUs.

We leverage GPU-specific features including `wgmma`, TMA (Tensor Memory Accelerator), warp specialization, and dynamic register reassignment. For BGMV, where computational intensity is low, we adopt thread collaborative execution instead of the heavier `wgmma` pipeline. For SGMV, we apply the swapping-AB transformation (computing $A^{\mathsf{T}}x^{\mathsf{T}}$) to align tensor shapes with hardware constraints, enabling efficient use of `wgmma.m64n8k16`. Additional optimizations include scheduling and persistent kernels. We omit further details due to space constraints and will release the implementation.

5.3 Layer-wise LoRA Loading

Due to finite GPU memory on the LoRA Server, cache misses are unavoidable and necessitate on-demand loading of LoRA weights. To prevent this from impacting the TTFT SLO, we design a layer-wise loading strategy that operates in concert with our hybrid parallel execution (Figure 10).

Rather than blocking until an entire adapter is loaded, we pipeline transfer and computation at layer granularity (loading including all experts, ranging from tens to hundreds of MB): LoRA computation for Layer 1 begins as soon as its weights arrive, while subsequent layers are fetched in the background across multiple GPUs, overlapped with ongoing LoRA execution. This latency is further amortized through

Table 3. Model and LoRA configurations. Instance #GPU denotes GPUs per LLM instance.

Model	#Layers	#Experts	Top-k	LoRA Rank	Instance #GPU
GPT-OSS-20B [24]	32	32	4	64	1
Qwen3-30B-A3B [35]	48	128	8	32	2
Mixtral-8x7B [16]	32	8	2	64	2
Scaled-MoE [52]	18	32	4	64	4
DBRX [36]	40	16	4	64	4

out-of-band signaling, whereby the scheduler instructs the LoRA Server to prefetch adapter weights before the LLM instance issues its first LoRA computation for Layer 1, often eliminating cold-start stalls entirely. With these optimizations, adapter loading does not affect the TTFT SLO under PCIe 5.0 (50 GB/s in our setup).

6 Evaluation

6.1 Evaluation Setup

Our experiments are conducted on a four-node cluster. Each node has four NVIDIA Hopper GPUs (96 GB), 96 CPU cores, 2 TB of host memory, and four 400 Gb/s InfiniBand NICs, each attached to a GPU. Intra-node GPUs communicate via 900 GB/s NVLink. We evaluate InfiniLoRA on five MoE models summarized in Table 3. For GPT-OSS-20B, we use 8 GPUs; for all other models, we use 16 GPUs. For Qwen3-30B-A3B, we use a reduced LoRA rank of 32 due to its fine-grained expert structure.

Workloads. Following prior work [3, 14, 21, 31], we simulate a multi-tenent workload where LoRA adapter popularity follows a Zipf distribution ($s = 1.2$), calibrated to production patterns in [53]. Unless otherwise specified, we use 512 adapters. Request arrivals follow a Poisson process with configurable rates, and input/output lengths are sampled from BurstGPT [37].

Metrics. We measure TTFT at the 95th percentile (P95) to capture tail latency from LoRA cache misses, and average TPOT for steady-state decoding performance. TTFT and TPOT SLOs are set to 0.25 s and 0.1 s respectively, consistent with previous work [52]. We treat each adapter as an independent service and define the system-wide *SLO Attainment Rate* [53] as the fraction of adapters whose requests meet SLOs in more than 90% of cases.

Methods Under Study. InfiniLoRA is implemented as a distributed system comprising a scheduler, a dedicated LoRA server, and multiple LLM instances built on vLLM [19]. LLM instances communicate with the LoRA server via a lightweight connector, and all LoRA kernels are implemented in CUDA. By default, the LoRA server and LLM instances reside on separate nodes connected via InfiniBand. We compare against S-LoRA [31] using its integrated vLLM implementation, which also serves as vLLM's official multi-LoRA serving backend. Both systems are allocated the same hardware budget and share identical LLM instance configurations; however, InfiniLoRA dedicates a portion of GPUs to the LoRA server and consequently runs fewer LLM instances. For S-LoRA, we allocate 50% of the remaining GPU memory (after loading base model weights) to LoRA cache and the other 50% to KV cache. Both systems use the same scheduler described in Section 2.2. To isolate scheduler-induced queueing effects, we additionally evaluate S-LoRA with a Shortest-Job-First scheduler that assumes oracle knowledge of output lengths, denoted S-LoRA w/ SJF. To further assess the impact of the LoRA cache ratio, we include a variant that allocates 40% and 60% of non-model GPU memory to the LoRA cache and KV cache respectively, denoted S-LoRA w/ Less LoRA. For both InfiniLoRA and all baselines, each LLM instance is pre-assigned a disjoint subset of adapters, determined by a greedy algorithm that targets load balance across instances. We also consider Toppings [21], but its CPU-based LoRA computation incurs prohibitive decode-time latency in production settings with multiple high-performance GPUs paired with limited CPU resources, so we exclude it. We omit scheduling-focused work [14, 53] as our approach is orthogonal to them, and under constrained cache capacity, no scheduling policy can generally outperform oracle SJF baseline.

6.2 Overall Performance Comparison

This section evaluates the performance of different frameworks across multiple models under varying load. For S-LoRA, given the LLM instance configurations in Table 3, the number of instances is determined by dividing the total num of GPUs by the GPUs required per instance. InfiniLoRA follows an SLO-driven provisioning strategy to first determine num of GPUs for LoRA Server and then assigning the remaining GPUs to LLM instances. In most experiments, InfiniLoRA uses 8 GPUs across two nodes for the LoRA Server, which provides a good balance between communication, computation, and synchronization. Due to testbed limitations, the number of LLM instances cannot be further increased even though the LoRA Server has spare capacity; we therefore evaluate scalability separately in Section 6.3.1. Overall, InfiniLoRA outperforms the baselines as shown in Figure 11.

InfiniLoRA achieves an average 3.05× increase in serviceable request rate across five models over S-LoRA while meeting both P95 TTFT and average TPOT SLOs. Moreover, InfiniLoRA improves the SLO attainment rate by an average of 54.0% and 53.1% compared to S-LoRA and S-LoRA w/ SJF, respectively. Against S-LoRA w/ Less LoRA, which allocates a smaller LoRA cache, the gains are even more pronounced: InfiniLoRA achieves a 4.56× higher serviceable request rate and improves SLO attainment by 60.6%. InfiniLoRA also improves throughput by 7.3% on average and up to 24.7% on DBRX compared to S-LoRA. Notably, InfiniLoRA attains higher throughput despite using fewer LLM instances, as the

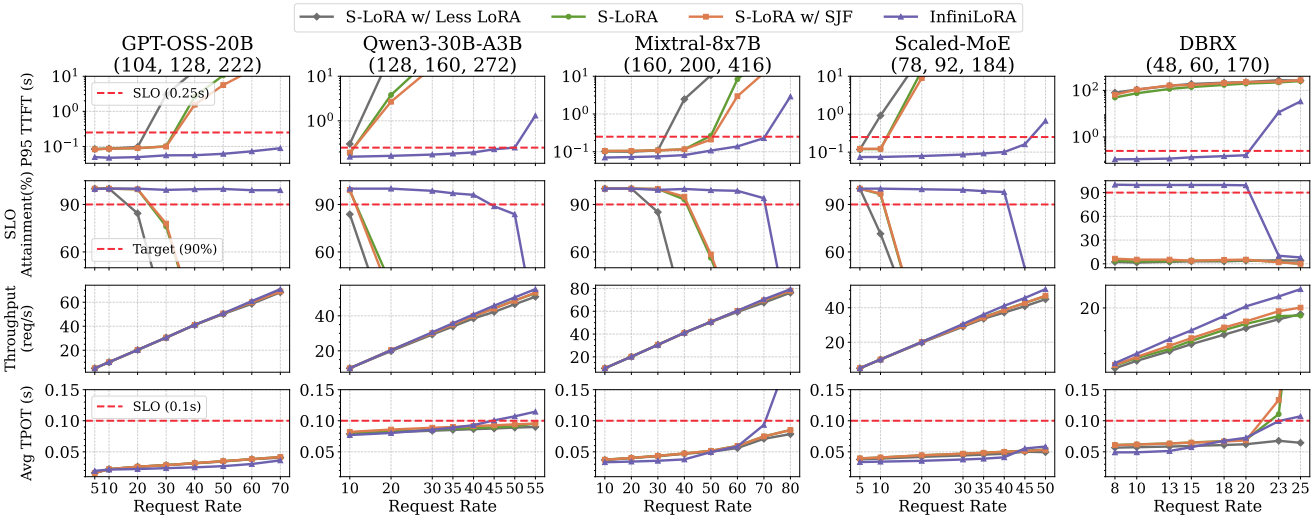
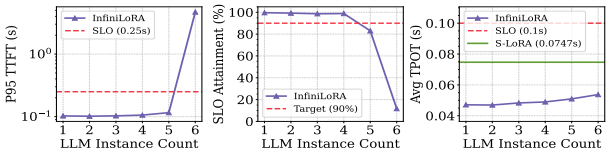
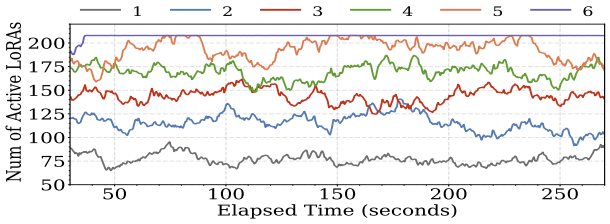


Figure 11. P95 TTFT, SLO attainment rate, throughput and average TPOT from top to bottom under varying loads. We compare InfiniLoRA against three baselines. The two values listed under each model name correspond to the LoRA cache capacity provided by S-LoRA w/ Less LoRA, S-LoRA (including w/ SJF) and InfiniLoRA, respectively.



(a) P95 TTFT, SLO attainment rate and average TPOT from left to right while scaling #LLM instances.



(b) The evolution of active LoRA counts over time.

Figure 12. Performance of scaling #LLM instances, configured with a request rate of 12 req/s per instance (keeping LoRA Server unchanged and using Mixtral-8x7B model).

larger effective batch size enabled by increased LoRA cache capacity improves GPU utilization. At high request rates on Mixtral, InfiniLoRA exhibits higher TPOT than S-LoRA. This occurs because both systems cache a large number of adapters, but under our fixed testbed resources InfiniLoRA runs with only half the number of LLM instances. With comparable throughput, each InfiniLoRA instance therefore handles nearly twice the request load, resulting in higher per-token latency.

6.3 Scalability Evaluation

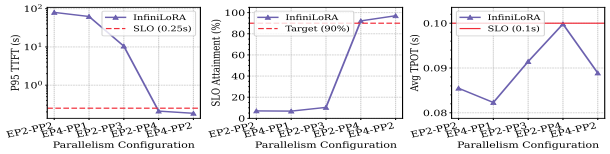
6.3.1 Scale with the Number of LLM Instances.

As shown in Figure 12, we scale the number of LLM instances from 1 to 6, allocating two GPUs per instance to serve Mixtral model, while fixing the LoRA Server at 4 GPUs. We proportionally increase the aggregate request rate from 12 to 72 req/s to maintain constant per-instance load. Average TPOT remains relatively stable, increasing by only 10.5% as the load on the LoRA Server gradually intensifies and still under SLO (0.1 s). Compared to S-LoRA (6 instances at 72 req/s), InfiniLoRA achieves lower TPOT because LoRA computation is largely overlapped with base-model execution, whereas S-LoRA executes LoRA serially. These results demonstrate that a 4-GPU LoRA Server provides sufficient computation throughput to serve 6 LLM instances, validating both the scalability and critical-path optimization of our design. However, both P95 TTFT and SLO attainment degrade sharply when the number of instances reaches six. As shown in Figure 12b, this degradation is caused by cache capacity saturation: under high request load, the number of active LoRA adapters reaches the LoRA Server's cache capacity, forcing subsequent requests to queue.

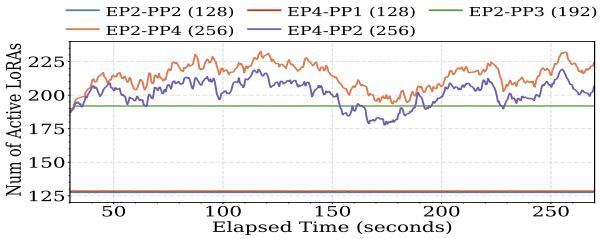
Insight1: in our disaggregated design, cache capacity is the primary scaling bottleneck when serving a large number of LLM instances, which motivates scaling the LoRA Server itself.

6.3.2 Scale with Server Resources.

With four LLM instances (each serving one Qwen3-30B-A3B model) and a fixed request rate of 35 req/s, we scale the LoRA Server by provisioning 4, 6, and 8 GPUs using five different parallelism configurations. As shown in Figure 13a, increasing LoRA Server resources significantly improves P95 TTFT and SLO attainment by expanding cache capacity. These observations



(a) P95 TTFT, SLO attainment rate and average TPOT from left to right using different LoRA Server parallelism configuration.



(b) The evolution of active LoRA counts over times. Numbers in parentheses denote the server's LoRA cache capacity.

Figure 13. Performance of scaling LoRA Server resources (we keep resources for LLM instances unchanged, using Qwen3-30B-A3B model and request rate=35 req/s).

are also consistent with the probabilistic model developed in Section 4.2: for LoRA cache capacities of 128, 192, and 256 in this setting, the model predicts immediate admission probabilities of 83.0%, 92.2%, and 100.0%, respectively, which closely matches the severely degraded P95 TTFT observed under smaller cache capacities. Under the 8-GPU configuration, a hybrid layout biased toward larger expert parallelism (i.e., EP_4-PP_2) achieves lower TPOT, as intra-node synchronization overhead is small while more GPUs are used to process each layer, aligning with our analysis in Section 4.1. Figure 13b shows the number of concurrently active adapters. For the 4-GPU and 6-GPU configurations, limited cache capacity caps the number of active adapters, directly leading to degraded tail TTFT and lower SLO attainment observed in Figure 13a.

Insight2: provisioning additional GPUs for LoRA Server effectively alleviates the cache capacity bottleneck, and that the choice of parallelism strategy further influences the efficiency of the disaggregated architecture.

6.4 Ablation study

We conduct an ablation study to quantify the impact of each optimization. Experiments use the Mixtral model with a fixed request rate of 25 req/s and 256 adapters in total. InfiniLoRA allocates 2 GPUs to the LoRA Server (cache capacity of 104) and 6 GPUs to three LLM instances, while S-LoRA uses all 8 GPUs to run four LLM instances (total cache capacity of 100); all other settings match the end-to-end evaluation. We start from a disaggregated baseline that separates LoRA adapters from the base model (`+disagg`), then add communication–computation overlap (`+overlap`), followed by

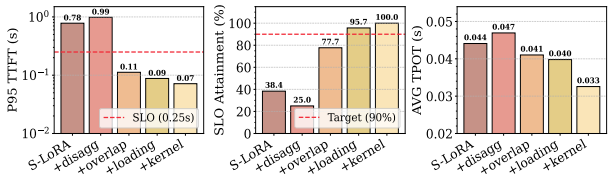


Figure 14. Ablation study for quantifying the effectiveness of individual optimization techniques. `+kernel` represents the fully optimized system with all techniques enabled.

layer-wise adapter loading (`+loading`). The full InfiniLoRA system further incorporates hardware-specialized kernels (`+kernel`).

As shown in the results, despite having a slightly larger cache capacity (104 over 100), `+disagg` alone increases tail TTFT from 0.78 s to 0.99 s, indicating that a naive disaggregated architecture actually degrades performance due to the additional communication overhead it introduces. By incrementally adding the remaining optimizations, InfiniLoRA reduces P95 TTFT by 11×, lowers average TPOT by 30%, and achieves a 100% SLO attainment rate, highlighting the complementary benefits of each technique.

Insight3: disaggregation alone is insufficient—its benefits are realized with critical-path optimization.

7 Related works

Request batching. Recent works like Punica [3] and S-LoRA [31] propose to batch requests with heterogeneous LoRA, typically utilizing on-demand loading. dLoRA [40] introduces similar merge/unmerge LoRA inference modes, also targeting at efficient request batching.

Request scheduling strategy. Chameleon [14] improves this by caching adapters in GPU memory and employs a multi-level scheduling queue. Cannikin [53] proposes a request scheduling strategy specifically tailored to optimize the lagger-SLO attainment. Since InfiniLoRA does not depend on request scheduling, these scheduling policies are orthogonal to our proposed system and can be readily integrated into our system with minimal modifications, serving as complementary components to enhance performance.

LoRA cache management. Toppings [21] attempts to leverage CPU cores for LoRA computation in prefill stages to address the prolong TTFT introduced by cache miss, but the limited host memory bandwidth makes this approach unsuitable for our low-latency decoding scenario. FASTLIBRA [47] also observes the impact of LoRA cache capacity on latency and it jointly manages LoRA adapters and KV caches within a unified HBM pool to reduce TTFT through dependency-aware eviction and swapping. LoRAServe [15] optimize LoRA placement among LLM instance clusters by

dynamically rebalancing adapters across GPUs. Despite LoRAServe's nominal resemblance to our approach, it targets a fundamentally different problem.

In conclusion, existing coupled architecture tightly binds LoRA adapters with base model, resulting in limited architectural flexibility and inability to address the challenge of insufficient LoRA cache capacity.

8 Conclusion

We introduce InfiniLoRA, a disaggregated serving system that decouples LoRA execution from base model inference to resolve scalability bottlenecks in multi-tenant scenarios. By leveraging a shared LoRA Server with parallelism-aware execution, SLO-driven provisioning, and critical-path optimizations, InfiniLoRA can flexibly scale LoRA cache capacity without interfering with the LLM inference pipeline. Our evaluation demonstrates that InfiniLoRA serves a 3.05× higher average request rate while satisfying latency SLOs, and improves the percentage of LoRA adapters satisfying the SLO requirement by 54.0% compared to existing systems.

References

[1] Anonymous. 2026. Understanding LoRA As Knowledge Memory: An Empirical Analysis. https://openreview.net/forum?id=i1Mi2R1TsU

[2] Chieh-Yun Chen, Zhonghao Wang, Qi Chen, Zhifan Ye, Min Shi, Yue Zhao, Yinan Zhao, Hui Qu, Wei-An Lin, Yiru Shen, Ajinkya Kale, Irfan Essa, and Humphrey Shi. 2025. MapReduce LoRA: Advancing the Pareto Front in Multi-Preference Optimization for Generative Models. arXiv:2511.20629 [cs.CV] https://arxiv.org/abs/2511.20629

[3] Lequn Chen, Zihao Ye, Yongji Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. 2023. Punica: Multi-Tenant LoRA Serving. arXiv:2310.18547 [cs.DC] https://arxiv.org/abs/2310.18547

[4] Yukang Chen, Shengju Qian, Haotian Tang, Xin Lai, Zhijian Liu, Song Han, and Jiaya Jia. 2024. LongLoRA: Efficient Fine-tuning of Long-Context Large Language Models. arXiv:2309.12307 [cs.CL] https://arxiv.org/abs/2309.12307

[5] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanjia Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. 2025. DeepSeek-V3 Technical Report. arXiv:2412.19437 [cs.CL] https://arxiv.org/abs/2412.19437

[6] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLORA: efficient finetuning of quantized LLMs. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 441, 28 pages.

[7] Kuntai Du, Bowen Wang, Chen Zhang, Yiming Cheng, Qing Lan, Hejian Sang, Yihua Cheng, Jiayi Yao, Xiaoxuan Liu, Yifan Qiao, Ion Stoica, and Junchen Jiang. 2025. PrefillOnly: An Inference Engine for Prefill-only Workloads in Large Language Model Applications. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles* (Lotte Hotel World, Seoul, Republic of Korea) *(SOSP '25)*. Association for Computing Machinery, New York, NY, USA, 399–414. doi:10.1145/3731569.3764834

[8] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. arXiv:2101.03961 [cs.LG] https://arxiv.org/abs/2101.03961

[9] Tianyu Gao, Alexander Wettig, Howard Yen, and Danqi Chen. 2025. How to Train Long-Context Language Models (Effectively). In *ACL*.

[10] Cunchen Hu, Heyang Huang, Junhao Hu, Jiang Xu, Xusheng Chen, Tao Xie, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. 2024. MemServe: Context Caching for Disaggregated LLM Serving with Elastic Memory Pool. arXiv:2406.17565 [cs.DC] https://arxiv.org/abs/2406.17565

[11] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Chenxi Wang, Jiang Xu, Shuang Chen, Hao Feng, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. 2025. ShuffleInfer: Disaggregate LLM Inference for Mixed Downstream Workloads. *ACM Trans. Archit. Code Optim.* 22, 2, Article 77 (July 2025), 24 pages. doi:10.1145/3732941

[12] Chenghao Hu, Yufei Kang, and Baochun Li. 2025. Communication-Efficient MoE Fine-Tuning with Locality-Aware Expert Placement. In *2025 IEEE 45th International Conference on Distributed Computing Systems (ICDCS)*. 166–176. doi:10.1109/ICDCS63083.2025.00025

[13] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*. https://openreview.net/forum?id=nZeVKeeFYf9

[14] Nikoleta Iliakopoulou, Jovan Stojkovic, Chloe Alverti, Tianyin Xu, Hubertus Franke, and Josep Torrellas. 2025. Chameleon: Adaptive Caching and Scheduling for Many-Adapter LLM Inference Environments. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25)*. Association for Computing Machinery, New York, NY, USA, 217–231. doi:10.1145/3725843.3756083

[15] Shashwat Jaiswal, Shrikara Arun, Anjaly Parayil, Ankur Mallick, Spyros Mastorakis, Alind Khare, Chloi Alverti, Renee St Amant, Chetan Bansal, Victor Rühle, and Josep Torrellas. 2025. Serving Heterogeneous LoRA Adapters in Distributed LLM Inference Systems. arXiv:2511.22880 [cs.DC] https://arxiv.org/abs/2511.22880

[16] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2024. Mixtral of Experts. arXiv:2401.04088 [cs.LG] https://arxiv.org/abs/2401.04088

[17] Rui Kong, Qiyang Li, Xinyu Fang, Qingtian Feng, Qingfeng He, Yazhu Dong, Weijun Wang, Yuanchun Li, Linghe Kong, and Yunxin Liu. 2024. LoRA-Switch: Boosting the Efficiency of Dynamic LLM Adapters via System-Algorithm Co-design. arXiv:2405.17741 [cs.AI] https://arxiv.org/abs/2405.17741

[18] Xiaoyu Kong, Jiancan Wu, An Zhang, Leheng Sheng, Hui Lin, Xiang Wang, and Xiangnan He. 2024. Customizing language models with instance-wise LoRA for sequential recommendation. In *Proceedings of the 38th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) *(NIPS '24)*. Curran Associates Inc., Red Hook, NY, USA, Article 3593, 24 pages.

[19] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.

[20] Hao Li, Chenghao Yang, An Zhang, Yang Deng, Xiang Wang, and Tat-Seng Chua. 2025. Hello Again! LLM-powered Personalized Agent for Long-term Dialogue. arXiv:2406.05925 [cs.CL] https://arxiv.org/abs/2406.05925

[21] Suyi Li, Yifan Qiao, Jiacheng Ma, Shan Yu, Haoran Ma, Ziming Liu, Hang Ren, Wenguang Chen, Yongwei Wu, Weimin Zheng, and Kang Chen. 2025. Toppings: Modular and Extensible Serverless Function Delivery at High Speed. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association, Seattle, WA. https://www.usenix.org/conference/atc25/presentation/li-suyi-toppings

[22] Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, Rui Kong, Yile Wang, Hanfei Geng, Jian Luan, Xuefeng Jin, Zilong Ye, Guanjing Xiong, Fan Zhang, Xiang Li, Mengwei Xu, Zhijun Li, Peng Li, Yang Liu, Ya-Qin Zhang, and Yunxin Liu. 2024. Personal LLM Agents: Insights and Survey about the Capability, Efficiency and Security. arXiv:2401.05459 [cs.HC] https://arxiv.org/abs/2401.05459

[23] Pak Markthub, Jim Dinan, Sreeram Potluri, and Seth Howell. 2022. *Improving Network Performance of HPC Systems Using NVIDIA Magnum IO NVSHMEM and GPUDirect Async*. NVIDIA. https://developer.nvidia.com/blog/improving-network-performance-of-hpc-systems-using-nvidia-magnum-io-nvshmem-and-gpudirect-async/

[24] OpenAI. 2025. gpt-oss-120b & gpt-oss-20b Model Card. arXiv:2508.10925 [cs.CL] https://arxiv.org/abs/2508.10925

[25] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2025. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture* (Buenos Aires, Argentina) *(ISCA '24)*. IEEE Press, 118–132. doi:10.1109/ISCA59077.2024.00019

[26] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. USENIX Association, Santa Clara, CA, 155–170. https://www.usenix.org/conference/fast25/presentation/qin

[27] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. 2022. DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale. arXiv:2201.05596 [cs.LG] https://arxiv.org/abs/2201.05596

[28] John Schulman and Thinking Machines Lab. 2025. LoRA Without Regret. *Thinking Machines Lab: Connectionism* (2025). doi:10.64434/tml.20250929 https://thinkingmachines.ai/blog/lora/.

[29] Sheng Shen, Le Hou, Yanqi Zhou, Nan Du, Shayne Longpre, Jason Wei, Hyung Won Chung, Barret Zoph, William Fedus, Xinyun Chen, Tu Vu, Yuexin Wu, Wuyang Chen, Albert Webson, Yunxuan Li, Vincent Zhao, Hongkun Yu, Kurt Keutzer, Trevor Darrell, and Denny Zhou. 2023. Mixture-of-Experts Meets Instruction Tuning:A Winning Combination for Large Language Models. arXiv:2305.14705 [cs.CL] https://arxiv.org/abs/2305.14705

[30] Zheyu Shen, Yexiao He, Ziyao Wang, Yuning Zhang, Guoheng Sun, Wanghao Ye, and Ang Li. 2025. *EdgeLoRA: An Efficient Multi-Tenant LLM Serving System on Edge Devices*. Association for Computing Machinery, New York, NY, USA, 138–153. https://doi.org/10.1145/3711875.3729141

[31] Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, Joseph E. Gonzalez, and Ion Stoica. 2023. S-LoRA: Serving Thousands of Concurrent LoRA Adapters. *arXiv preprint arXiv:2311.03285* (2023).

[32] Ge Shi, Hanieh Sadri, Qian Wang, Yu Zhang, Ying Xiong, Yong Zhang, and Zhenan Fan. 2025. ExpertWeave: Efficiently Serving Expert-Specialized Fine-Tuned Adapters at Scale. arXiv:2508.17624 [cs.DC] https://arxiv.org/abs/2508.17624

[33] Weijia Shi, Sewon Min, Maria Lomeli, Chunting Zhou, Margaret Li, Xi Victoria Lin, Noah A. Smith, Luke Zettlemoyer, Wen tau Yih, and Mike Lewis. 2024. In-Context Pretraining: Language Modeling Beyond Document Boundaries. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=LXVswInHOo

[34] Xiao Shi, Jiangsu Du, Zhiguang Chen, and Yutong Lu. 2025. AuLoRA: Fine-Grained Loading and Computation Orchestration for Efficient LoRA LLM Serving. In *2025 IEEE 43rd International Conference on Computer Design (ICCD)*. 277–284. doi:10.1109/ICCD65941.2025.00046

[35] Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] https://arxiv.org/abs/2505.09388

[36] The Mosaic Research Team. 2024. Introducing DBRX: A New State-of-the-Art Open LLM. https://www.databricks.com/blog/introducing-dbrx-new-state-art-open-llm. Accessed: 2026-01-21.

[37] Yuxin Wang, Yuhan Chen, Zeyu Li, Xueze Kang, Yuchu Fang, Yeju Zhou, Yang Zheng, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. 2025. BurstGPT: A Real-World Workload Dataset to Optimize LLM Serving Systems. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '25)*. ACM, Toronto, ON, Canada. doi:10.1145/3711896.3737413

[38] Yu Wang, Dmitry Krotov, Yuanzhe Hu, Yifan Gao, Wangchunshu Zhou, Julian McAuley, Dan Gutfreund, Rogerio Feris, and Zexue He. 2025. M+: Extending MemoryLLM with Scalable Long-Term Memory. arXiv:2502.00592 [cs.CL] https://arxiv.org/abs/2502.00592

[39] Zhengbo Wang, Jian Liang, Ran He, Zilei Wang, and Tieniu Tan. 2025. LoRA-Pro: Are Low-Rank Adapters Properly Optimized?. In *The Thirteenth International Conference on Learning Representations (ICLR)*.

[40] Bingyang Wu, Ruidong Zhu, Zili Zhang, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. dLoRA: Dynamically Orchestrating Requests and Adapters for LoRA LLM Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 911–927. https://www.usenix.org/conference/osdi24/presentation/wu-bingyang

[41] Shuaipeng Wu, Yanying Lin, Shijie Peng, Wenyan Chen, Chong Ma, Min Shen, Le Chen, Chengzhong Xu, and Kejiang Ye. 2025. Rock: Serving Multimodal Models in Cloud with Heterogeneous-Aware Resource Orchestration for Thousands of LoRA Adapters. In *2025 IEEE International Conference on Cluster Computing (CLUSTER)*. 1–13. doi:10.1109/CLUSTER59342.2025.11186463

[42] Lingnan Xia and Hua Ma. 2024. Enhancing LoRA Model Serving Capacity via Adaptive Operator Scheduling for Multi-Tenancy on GPU. *IEEE Access* 12 (2024), 160441–160449. doi:10.1109/ACCESS.2024.3483250

[43] Yifei Xia, Fangcheng Fu, Wentao Zhang, Jiawei Jiang, and Bin Cui. 2024. Efficient multi-task LLM quantization and serving for multiple LoRA adapters. In *Proceedings of the 38th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) *(NIPS '24)*. Curran Associates Inc., Red Hook, NY, USA, Article 2034, 29 pages.

[44] Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Junxian Guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. 2024. DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads. arXiv:2410.10819 [cs.CL] https://arxiv.org/abs/2410.10819

[45] Yahe Yang, Chunliang Tao, and Xiaojing Fan. 2025. LoRA-LiteE: A Computationally Efficient Framework for Chatbot Preference-Tuning. arXiv:2411.09947 [cs.CL] https://arxiv.org/abs/2411.09947

[46] Hang Zhang, Jiuchen Shi, Yixiao Wang, Quan Chen, Yizhou Shan, and Minyi Guo. 2025. Improving the Serving Performance of Multi-LoRA

Large Language Models via Efficient LoRA and KV Cache Management. arXiv:2505.03756 [cs.AR] https://arxiv.org/abs/2505.03756

[47] Hang Zhang, Jiuchen Shi, Yixiao Wang, Quan Chen, Yizhou Shan, and Minyi Guo. 2025. Improving the Serving Performance of Multi-LoRA Large Language Models via Efficient LoRA and KV Cache Management. arXiv:2505.03756 [cs.AR] https://arxiv.org/abs/2505.03756

[48] Tianyu Zhang, Peng Zhang, Yusong Gao, and Yun Zhang. 2025. Together with SGLang: Best Practices for Serving DeepSeek-R1 on H20-96G. https://lmsys.org/blog/2025-09-26-sglang-ant-group/. LMSYS Org Blog.

[49] You Zhang, Jin Wang, Liang-Chih Yu, Dan Xu, and Xuejie Zhang. 2024. Personalized LoRA for Human-Centered Text Understanding. *Proceedings of the AAAI Conference on Artificial Intelligence* 38, 17 (Mar. 2024), 19588–19596. doi:10.1609/aaai.v38i17.29931

[50] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) *(OSDI'24)*. USENIX Association, USA, Article 11, 18 pages.

[51] Changhai Zhou, Yuhua Zhou, Shiyang Zhang, Yibin Wang, and Zekai Liu. 2025. Dynamic Operator Optimization for Efficient Multi-Tenant LoRA Model Serving. *Proceedings of the AAAI Conference on Artificial Intelligence* 39, 21 (Apr. 2025), 22910–22918. doi:10.1609/aaai.v39i21.34453

[52] Ruidong Zhu, Ziheng Jiang, Chao Jin, Peng Wu, Cesar A. Stuardo, Dongyang Wang, Xinlei Zhang, Huaping Zhou, Haoran Wei, Yang Cheng, Jianzhe Xiao, Xinyi Zhang, Lingjun Liu, Haibin Lin, Li-Wen Chang, Jianxi Ye, Xiao Yu, Xuanzhe Liu, Xin Jin, and Xin Liu. 2025. MegaScale-Infer: Efficient Mixture-of-Experts Model Serving with Disaggregated Expert Parallelism. In *Proceedings of the ACM SIGCOMM 2025 Conference* (São Francisco Convent, Coimbra, Portugal) *(SIGCOMM '25)*. Association for Computing Machinery, New York, NY, USA, 592–608. doi:10.1145/3718958.3750506

[53] Ruidong Zhu, Ziyue Jiang, Zhi Zhang, Xin Liu, Xuanzhe Liu, and Xin Jin. 2025. Cannikin: No Lagger of SLO in Concurrent Multiple LoRA LLM Serving. *IEEE Transactions on Parallel and Distributed Systems* 36, 9 (2025), 1972–1984. doi:10.1109/TPDS.2025.3590014

A Appendix

A.1 Additional Scalability Test

A.1.1 Scale with different LoRA load. We further evaluate InfiniLoRA's performance sensitivity to varying LoRA workload characteristics. Specifically, we sweep the adapter access distribution skewness s (fixing the pool size at 512) and the total number of adapters (fixing the skew parameter $s = 1.2$), as shown in Figure 15. We only compared the experiments on Mixtral 8x7B model and request rate of 70 req/s; the other configurations were the same as in the end-to-end evaluation. For scenarios with high locality ($s = 1.5$) or smaller adapter pools (256), we downscale the LoRA Server resources to 4-GPUs following the provisioning policy detailed in Section 4.2, and we use 8-GPUs for other configurations. The results demonstrate that InfiniLoRA successfully meets SLOs across the majority of configurations. The only exception occurs with a large pool of 1024 adapters. In this regime, we believe that provisioning additional resources to LoRA Server is necessary. Doing so not only recovers SLO attainment rate but also enables supporting a larger number of concurrent LLM instances.

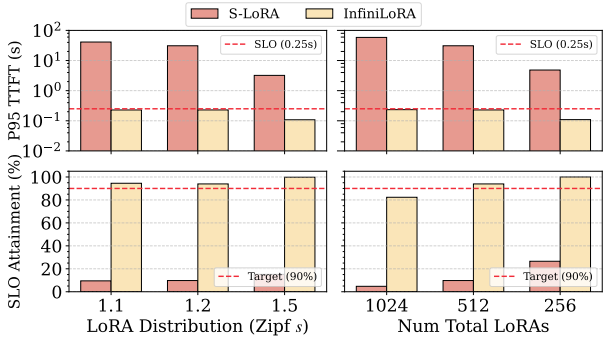


Figure 15. Scalability under varying LoRA popularity distributions and adapter counts.

A.1.2 Scale with batch size. We evaluate LoRA Server's processing latency across varying task sizes as shown in Figure 16. Our experimental setup consists of a 4-GPU LoRA Server serving two types of LLM instances: a Mixtral 8x7B model (2 GPUs) or a Scaled MoE model (4 GPUs). We observe that communication latency scales linearly with batch size, as it is strictly bound by the send/receive bandwidth of the LLM instance's NICs. In contrast, LoRA computation time increases sub-linearly with batch size. This behavior stems from the power-law distribution of LoRA popularity: linearly increasing the batch size does not lead to a proportional rise in the number of distinct LoRA invocations, which is the primary driver of computation latency due to the memory-bound nature of LoRA computation. Consequently, when the number of tokens per iteration reaches 1024 or 4096 (corresponding to a batch size of 512 for Mixtral 8x7B or 1024 for Scaled MoE), network bandwidth becomes the dominant bottleneck, leaving the LoRA Server's compute resources underutilized. However, such large batch sizes are rarely seen in production due to KV cache capacity constraints and strict TPOT requirements during decoding. Therefore, despite the theoretical hardware underutilization in these extreme scenarios, InfiniLoRA maintains high efficiency under realistic end-to-end workloads.

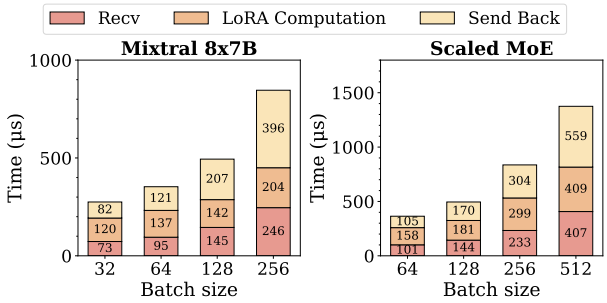


Figure 16. Per-iteration latency breakdown on the LoRA Server under varying load.

A.1.3 Scale with network bandwidth. Finally, we investigate InfiniLoRA's sensitivity to the underlying interconnect bandwidth and latency. We compare the default inter-node deployment (via InfiniBand) against a collocated deployment where the LoRA Server and LLM instances reside on a single 8-GPU node connected via full-mesh NVLink. We use a total of 256 adapters and fix the request rate at 30 req/s. The LoRA Server is configured with 2 GPUs, while each LLM instance consists of 2 GPUs hosting a Mixtral 8x7B model. We vary the number of LLM instances from 1 to 3. All other settings remain consistent with the end-to-end evaluation.

As shown in Figure 17, the NVLink-based configuration benefits significantly from the lower latency and higher bandwidth of intra-node communication. Compared to the InfiniBand deployment, NVLink reduces average TPOT by 14.6% and improves the SLO attainment rate by up to 46.1% across varying numbers of LLM instances. This confirms that while InfiniLoRA is designed for disaggregated clusters, it can readily exploit faster interconnects to further improve serving performance.

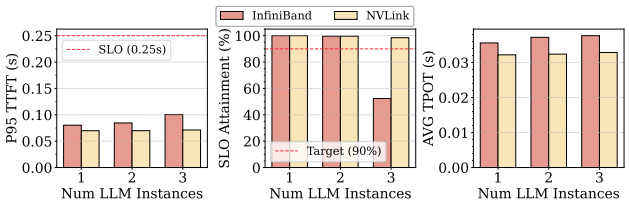


Figure 17. Impact of interconnect bandwidth on InfiniLoRA's serving performance: NVLink vs. InfiniBand.

A.2 Microbenmark

A.2.1 LoRA data layout. We conduct a microbenchmark to analyze the latency breakdown of LoRA execution under different parallelization strategies, as detailed in Table 4. The LoRA Server is deployed on 8 GPUs with four parallelism configurations: pipeline parallel (EP_1-PP_8), expert parallel (EP_8-PP_1), and two hybrid configurations (EP_2-PP_4 and EP_4-PP_2) (Data Parallelism is omited for its significant defects). Workloads are generated by an LLM instance running the Mixtral 8x7B model at two representative batch sizes.

The EP_1-PP_8 configuration assumes an ideal scenario in which multiple LLM instances are naturally distributed across pipeline stages without interference. However, it fails to achieve optimal performance, primarily due to high LoRA execution latency and the inherent instability of asynchronous pipelines. For the remaining three configurations involving expert parallelism, communication overhead remains relatively constant, as it is bottlenecked by the NIC bandwidth of the LLM instance. The pure expert-parallel setup (EP_8-PP_1) exhibits significant diminishing returns: distributing LoRA execution across too many GPUs paradoxically increases both computation and communication latencies.

Among the hybrid configurations, EP_4-PP_2 offers a more favorable trade-off than EP_2-PP_4. The latter relies heavily on perfect load balancing across a deep pipeline, making it highly susceptible to performance degradation caused by inter-execution interference when multiple LLM instances share the same pipeline stage. In contrast, EP_4-PP_2 maximizes per-stage processing capability for LoRA execution while using two pipeline stages to sustain throughput and avoid a larger synchronization scope. We empirically validate this in Figure 18, where the end-to-end performance comparison confirms the superiority of the EP_4-PP_2 configuration.

Table 4. Latency breakdown of LoRA execution and base MoE computation under varying parallelism configurations (µs).

EP_x-PP_y	Batch Size = 128				Batch Size = 256			
	Recv	LoRA	Send	MoE	Recv	LoRA	Send	MoE
EP_1-PP_8	243	342	384	493	527	526	734	762
EP_2-PP_4	155	212	221	493	279	315	402	764
EP_4-PP_2	145	142	207	492	246	204	396	763
EP_8-PP_1	173	163	255	493	310	205	441	763

A.2.2 LoRA kernels. We evaluate the performance of our proposed LoRA computation kernels against two state-of-the-art baselines, Punica [3] and S-LoRA [31], as shown in Figure 19. The workload comprises 512 distinct LoRA adapters with a batch size of 1024, where LoRA invocation probability follows a Zipf distribution ($s = 1.2$).

Overall, InfiniLoRA's kernels demonstrate significant advantages over the baselines in both latency and GPU memory bandwidth utilization. Specifically, InfiniLoRA-BGMV excels during the shrink phase but exhibits performance degradation in the expand phase, primarily due to the larger volume of data written back in the latter. In contrast, InfiniLoRA-SGMV maintains consistent performance across both phases. Furthermore, InfiniLoRA-SGMV achieves lower latency than InfiniLoRA-BGMV by aggregating tokens that share the same LoRA adapter into a single GEMM operation, thereby reducing memory bandwidth consumption effectively.

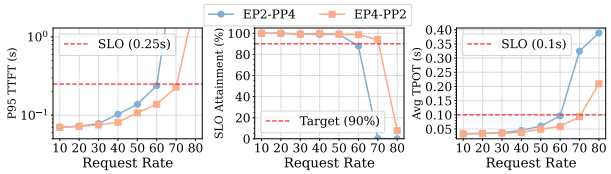


Figure 18. End-to-end performance comparison of two LoRA data layouts (EP_2-PP_4 and EP_4-PP_2) under the same LoRA cache capacity.

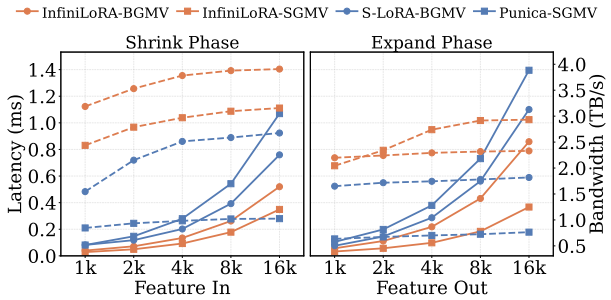


Figure 19. Characterization of latency and bandwidth for distinct LoRA kernels across shrink/expand phases. Dashed lines represent bandwidth and solid lines indicate latency.